VHDL Lab Assignment 2 Report

Five Latches and Flip-Flops

- Logisim Gate Circuits, VHDL Code, and Questa Simulation

1. Overview of the Five Circuits

In this lab, five basic 1-bit memory elements were drawn at the gate level in Logisim, and the same elements were written in VHDL to check whether the two implementations produce identical outputs. In each screenshot, the upper part is the gate-level circuit and the lower part is the VHDL entity imported into Logisim as a block; pink circles mark the inputs changed at that step. The same VHDL code was then compiled with vcom and simulated with vsim in Questa Altera Starter FPGA Edition 2025.2, and the three results (gate circuit, Logisim VHDL block, Questa waveform) were compared.

No.	Circuit	Logisim Circuit Structure	VHDL File	Key Behavior
1	RS latch	Two NOR gates with outputs cross-coupled to each other's inputs	rs_latch.vhd	S=1 Set, R=1 Reset, hold when both are 0, both 1 is forbidden
2	D latch	Latch built from four NAND gates taking D and en	d_latch.vhd	Q follows D while en=1, holds the previous value when en=0
3	D flip-flop	Two D latches + NOT gate in a Master-Slave configuration	d_ff.vhd	Stores D only at the instant clk changes 0→1
4	JK flip-flop	NAND gates form the d_next logic, with outputs fed back to the inputs	jk_ff.vhd	On the rising edge: 00 hold, 01 Reset, 10 Set, 11 toggle
5	D flip-flop with asynchronous Set/Clear	D flip-flop with clock-independent set and clr paths added	d_ff_async.vhd	clr=1 gives 0 immediately, set=1 gives 1 immediately, Clear has priority when both are 1

- Circuits 1 and 2 (latches) are level-triggered elements: the input is passed to the output for as long as enable is active.
- Circuits 3, 4, and 5 (flip-flops) are edge-triggered elements: only the input present at the rising clock edge is stored.
- Circuit 5 adds asynchronous set/clr inputs that override the clock; in VHDL the priority is set by the sensitivity list and the order of the if checks.
- In Questa, the design was loaded with vlib work → vcom file.vhd → vsim work.entity, signals were registered with add wave, and force/run commands were issued in 10 ns steps. For the flip-flops, a rising edge was made by changing the inputs first and then raising clk 0→1.

2. RS Latch (rs_latch)

The most basic latch: two NOR gates are cross-coupled so that the S and R inputs Set/Reset and store one bit.

■ VHDL Code

[rs_latch.vhd]

```
library ieee;
use ieee.std_logic_1164.all;

entity rs_latch is
    port (
        s, r : in std_logic;
        q, q_bar : out std_logic
    );
end rs_latch;

architecture gate_level of rs_latch is
    signal qi, qbi : std_logic;
begin
    qi  <= not (r or qbi);
    qbi <= not (s or qi);
    q <= qi;
    q_bar <= qbi;
end gate_level;
```

■ Code Explanation

- The two NOR gates are described directly with two concurrent signal assignments, without a process.
- The feedback structure, in which qi and qbi feed each other's inputs, provides the memory function.
- Because q_bar is the output of the second NOR, both q and q_bar become 0 when S=R=1 (forbidden input).

■ Schematic and Results

In the upper part of the schematic, the output of the NOR fed by r and the output of the NOR fed by s each return to the input of the other gate; the lower part is the rs_latch VHDL block receiving the same inputs. The inputs were changed in the order Set → hold → Reset → hold → forbidden input, and q and q_bar of the two implementations were compared.

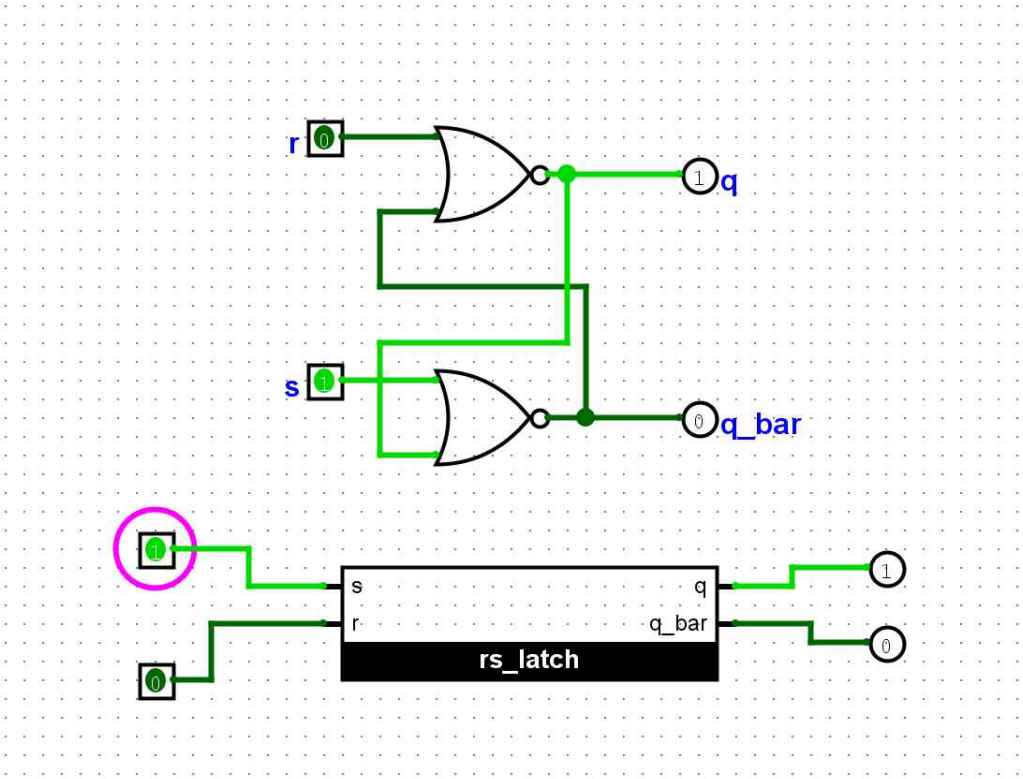


Figure 1-1. S=1, R=0 → q=1, q_bar=0 (Set). Both the gate circuit and the rs_latch block give q=1

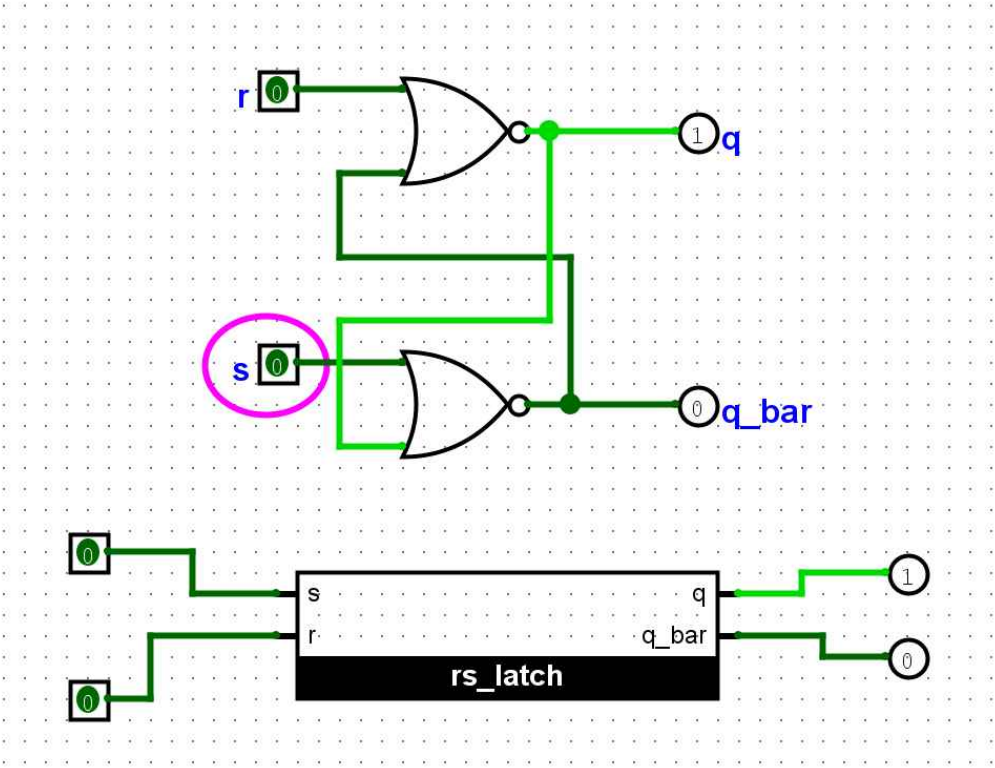


Figure 1-2. S=0, R=0 → q=1 held. The previous value 1 is remembered even after both inputs are lowered

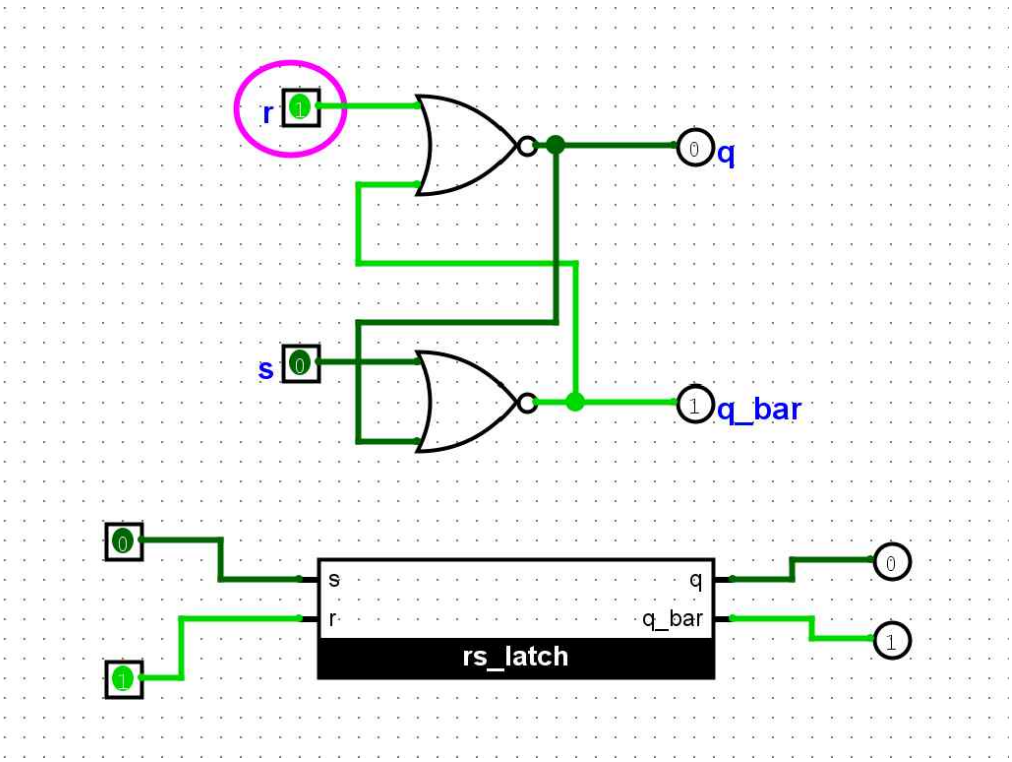


Figure 1-3. S=0, R=1 → q=0, q_bar=1 (Reset)

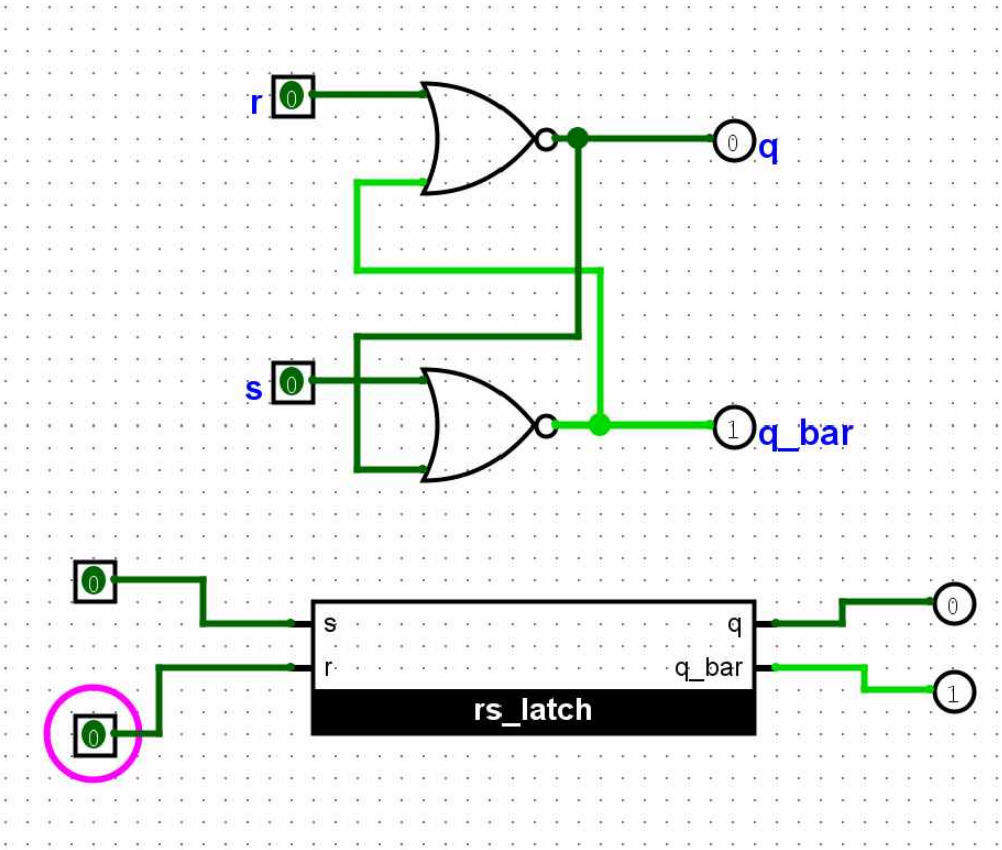


Figure 1-4. S=0, R=0 → q=0 held. The 0 from the Reset is remembered

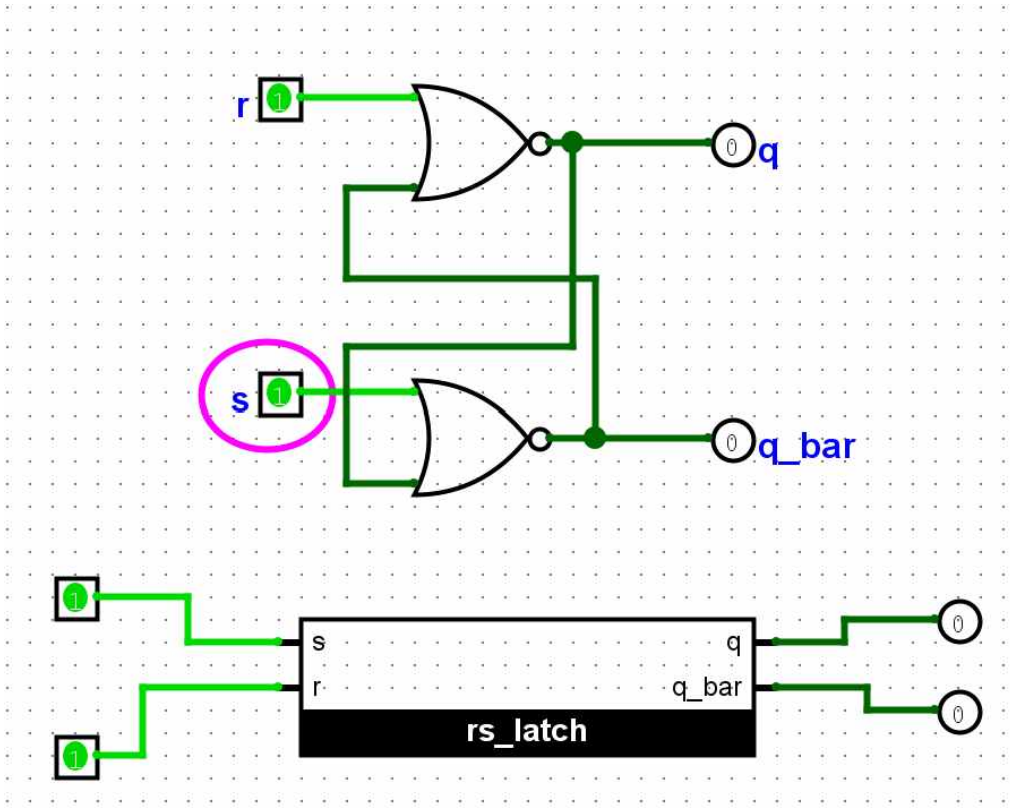


Figure 1-5. S=1, R=1 → q=0, q_bar=0 (forbidden input). The two outputs are no longer complementary

In all five steps the outputs of the gate circuit and the VHDL block were identical, confirming the memory behavior (previous value held at S=R=0) and the forbidden state at S=R=1.

■ Questa Compilation and Simulation

The same rs_latch.vhd was compiled with vcom in Questa Altera Starter FPGA Edition 2025.2 and simulated in vsim, applying the same input sequence as the Logisim experiment with force commands. The script entered in the Transcript window is shown below.

[rs_latch_sim.do]

```
add wave sim:/rs_latch/s sim:/rs_latch/r sim:/rs_latch/q sim:/rs_latch/q_bar
force s 1
force r 0
run 10 ns
force s 0
run 10 ns
force r 1
run 10 ns
```

```
force r 0
run 10 ns
force s 1
force r 1
run 10 ns
wave zoom full
```

```
VSIM 18> vcom {C:\VHDL_Assignment\2번실습\rs_latch.vhd}
# Questa Altera Starter FPGA Edition-64 vcom 2025.2 Compiler 2025.05 May 31 2025
# Start time: 09:29:19 on Nov 11,2026
# vcom -reportprogress 300 C:\VHDL_Assignment\2¹ø½Ç½À\rs_latch.vhd
# ** Note: (vcom-220) 'C:/Users/gahyu/AppData/Local/quartus/Q1_rs_latch.mpf' is used as the ini file.
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Compiling entity rs_latch
# -- Compiling architecture gate_level of rs_latch
# End time: 09:29:19 on Nov 11,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
```

Figure 1-6a. Compilation result of rs_latch.vhd (Errors: 0, Warnings: 0)

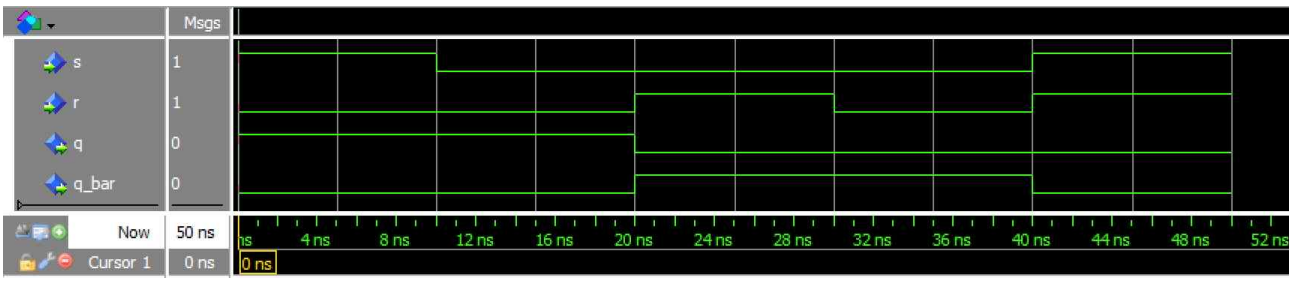


Figure 1-6b. Questa waveform, top to bottom: s, r, q, q_bar. Each interval is 10 ns

Interval	Inputs	q	Observed behavior
0~10 ns	s=1, r=0	1	Set. q=1, q_bar=0 (same as Figure 1-1)
10~20 ns	s=0, r=0	1	Hold. 1 is remembered after the inputs are lowered (Figure 1-2)
20~30 ns	s=0, r=1	0	Reset. q=0, q_bar=1 (Figure 1-3)
30~40 ns	s=0, r=0	0	Hold. The 0 from the Reset is remembered (Figure 1-4)
40~50 ns	s=1, r=1	0	Forbidden input. Both q and q_bar are 0 (Figure 1-5)

The q and q_bar values in all five intervals matched the results of the Logisim gate circuit and VHDL block (Figures 1-1 to 1-5). The three implementations exhibit the same RS latch function.

3. D Latch (d_latch)

A latch built from four NAND gates. While en=1 the output Q follows the input D, and when en=0 the last value is held.

■ VHDL Code

[d_latch.vhd]

```
library ieee;
use ieee.std_logic_1164.all;

entity d_latch is
    port (
        d, en : in std_logic;
        q, q_bar : out std_logic
    );
end d_latch;

architecture rtl of d_latch is
    signal qi : std_logic;
begin
    process(d, en)
    begin
        if en = '1' then
            qi <= d;
        end if;
    end process;

    q <= qi;
    q_bar <= not qi;
end rtl;
```

■ Code Explanation

- process(d, en) has no clock; it contains only if en = '1' then qi <= d; with no else branch.
- Because there is no else, qi keeps its previous value when en=0, which makes this a level-sensitive latch.
- q_bar is generated as the inverse of qi, so it always stays complementary to q.

■ Schematic and Results

In the upper part of the schematic, S_bar = NAND(D, en) and R_bar = NAND(D_bar, en) feed the NAND latch (Q, Q_bar) behind them; the lower part is the d_latch VHDL block. D was changed while opening and closing en to check when the output follows and when it holds.

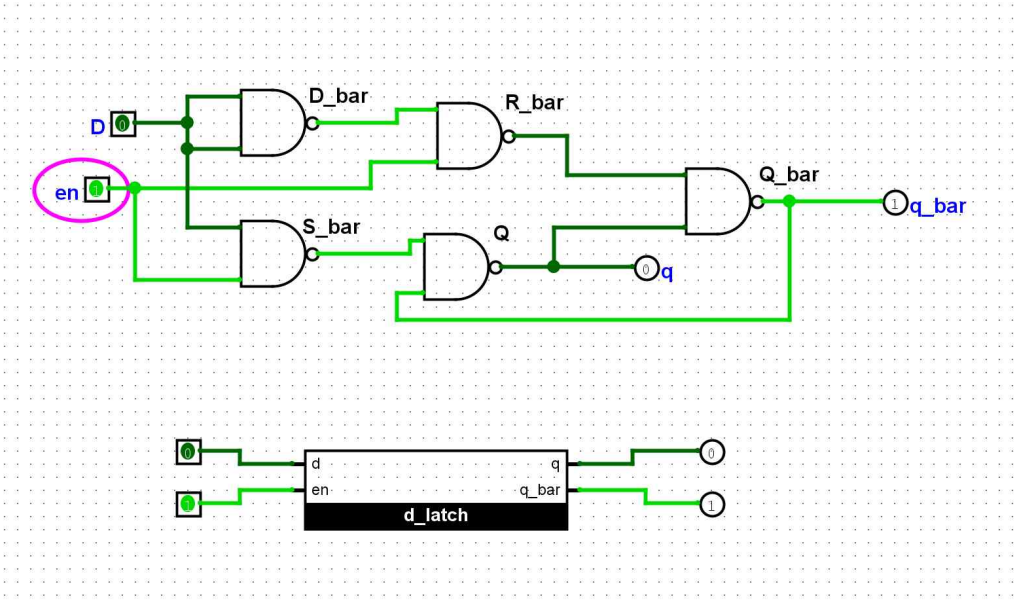


Figure 2-1. en=1, D=0 → q=0. With en open, D=0 is reflected at the output immediately

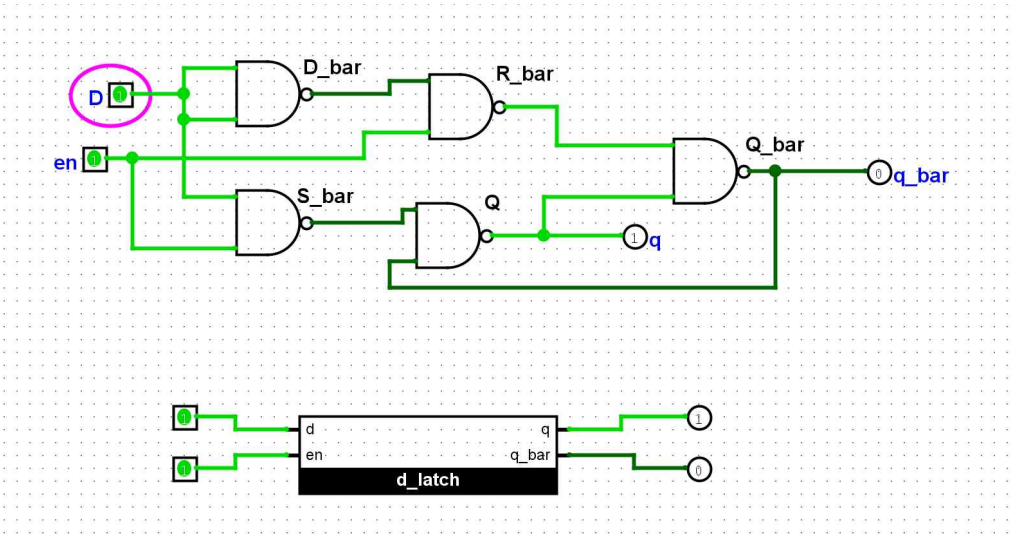


Figure 2-2. en=1, D=1 → q=1. The latch is open, so Q follows D

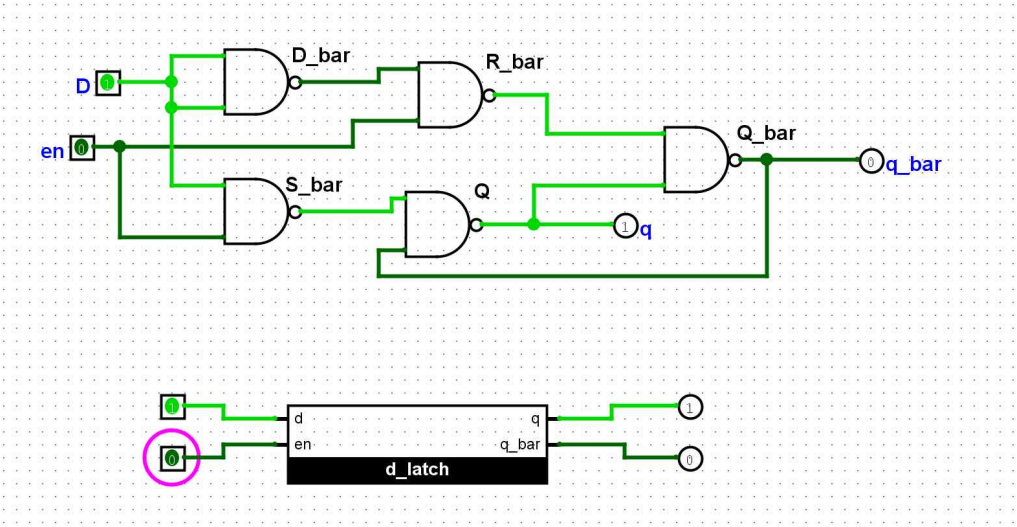


Figure 2-3. en=0, D=1 → q=1 held. The D value at the moment en is lowered is stored

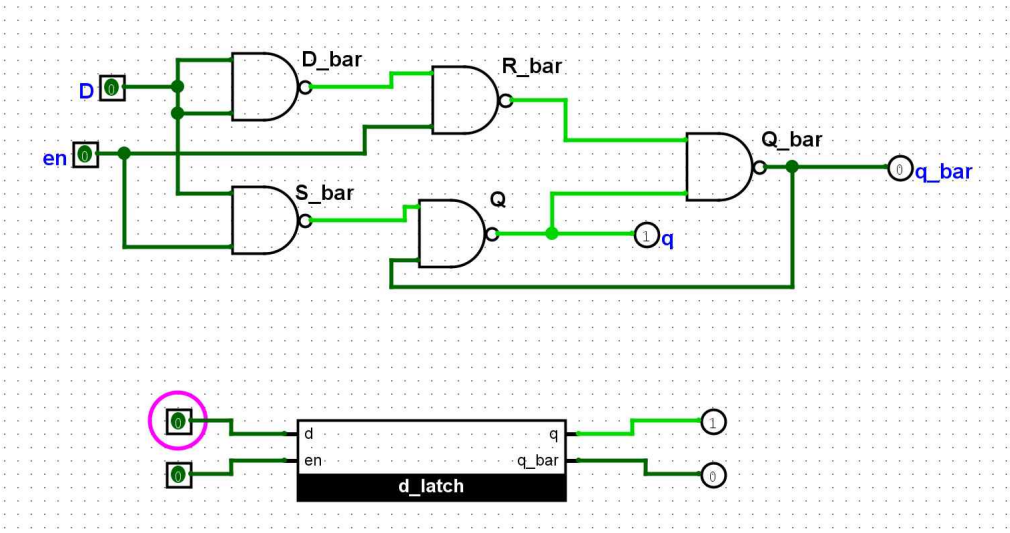


Figure 2-4. en=0, D=0 → q=1 held. While closed, the output does not change even if D changes

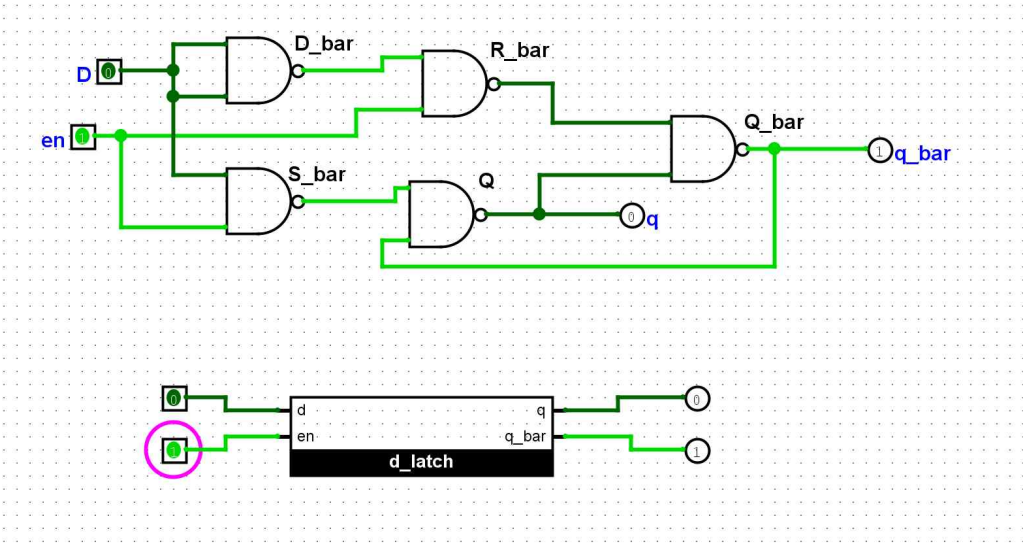


Figure 2-5. en=1, D=0 → q=0. Once reopened, the current D=0 is reflected

During en=1, Q immediately followed D; during en=0, Q did not change even when D changed. The outputs of the gate circuit and the VHDL block agreed in every step.

■ Questa Compilation and Simulation

The same d_latch.vhd was compiled with vcom in Questa Altera Starter FPGA Edition 2025.2 and simulated in vsim, applying the same input sequence as the Logisim experiment with force commands. The script entered in the Transcript window is shown below.

[d_latch_sim.do]

```
add wave sim:/d_latch/d sim:/d_latch/en sim:/d_latch/q sim:/d_latch/q_bar
force en 1
force d 0
run 10 ns
force d 1
run 10 ns
force en 0
run 10 ns
force d 0
run 10 ns
force en 1
run 10 ns
wave zoom full
```

```
VSIM 63> vcom {C:\VHDL_Assignment\test2\d_latch.vhd}
# Questa Altera Starter FPGA Edition-64 vcom 2025.2 Compiler 2025.05 May 31 2025
# Start time: 09:41:31 on Nov 11,2026
# vcom -reportprogress 300 C:\VHDL_Assignment\test2\d_latch.vhd
# ** Note: (vcom-220) 'C:/Users/gahyu/AppData/Local/quartus/Q1_rs_latch.mpf' is used as the ini file.
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Compiling entity d_latch
# -- Compiling architecture rtl of d_latch
# End time: 09:41:31 on Nov 11,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
```

Figure 2-6a. Compilation result of d_latch.vhd (Errors: 0, Warnings: 0)

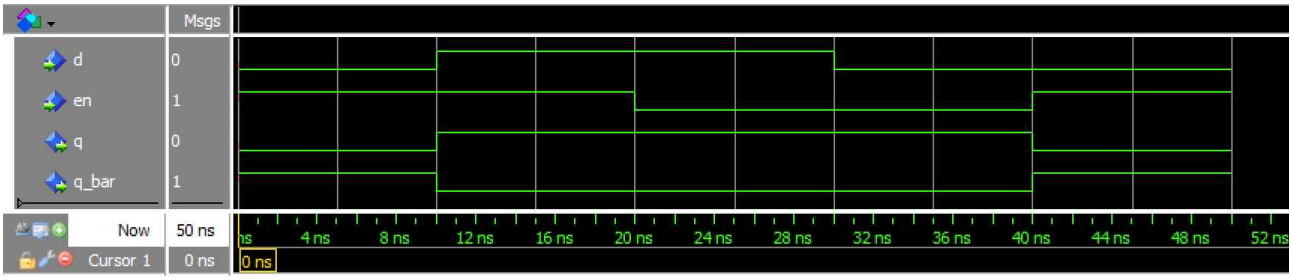


Figure 2-6b. Questa waveform, top to bottom: d, en, q, q_bar. Each interval is 10 ns

Interval	Inputs	q	Observed behavior
0~10 ns	en=1, d=0	0	Open. d=0 is reflected immediately (Figure 2-1)
10~20 ns	en=1, d=1	1	Open. q follows d (Figure 2-2)
20~30 ns	en=0, d=1	1	Closed. d=1 at the moment en is lowered is stored (Figure 2-3)
30~40 ns	en=0, d=0	1	Closed. q unchanged even though d changes (Figure 2-4)
40~50 ns	en=1, d=0	0	Reopened. Current d=0 is reflected (Figure 2-5)

q followed d only during the en=1 intervals, and changes of d were ignored during en=0. All five intervals match the Logisim results in Figures 2-1 to 2-5.

4. D Flip-Flop (d_ff)

Two D latches from circuit 2 are connected in a Master-Slave configuration with a clock inverted by a NOT gate, giving a flip-flop that stores D only on the rising clock edge.

■ VHDL Code

[d_ff.vhd]

```
library ieee;
use ieee.std_logic_1164.all;

entity d_ff is
    port (
        d, clk : in std_logic;
        q, q_bar : out std_logic
    );
end d_ff;

architecture rtl of d_ff is
    signal qi : std_logic;
begin
    process(clk)
    begin
        if rising_edge(clk) then
            qi <= d;
        end if;
    end process;

    q <= qi;
    q_bar <= not qi;
end rtl;
```

■ Code Explanation

- if rising_edge(clk) then qi <= d; stores d only on the rising edge of the clock.
- The edge behavior built from two latches and a NOT gate in the gate circuit is expressed in VHDL with the single rising_edge() call.
- d is sampled only at the edge, so it is not included in the sensitivity list.

■ Schematic and Results

In the upper part of the schematic, the master latch (en = NOT clk) and the slave latch (en = clk) are connected in series; the lower part is the d_ff VHDL block. When clk=0 the master accepts D, and at the instant clk changes 0→1 the slave opens and the output q changes.

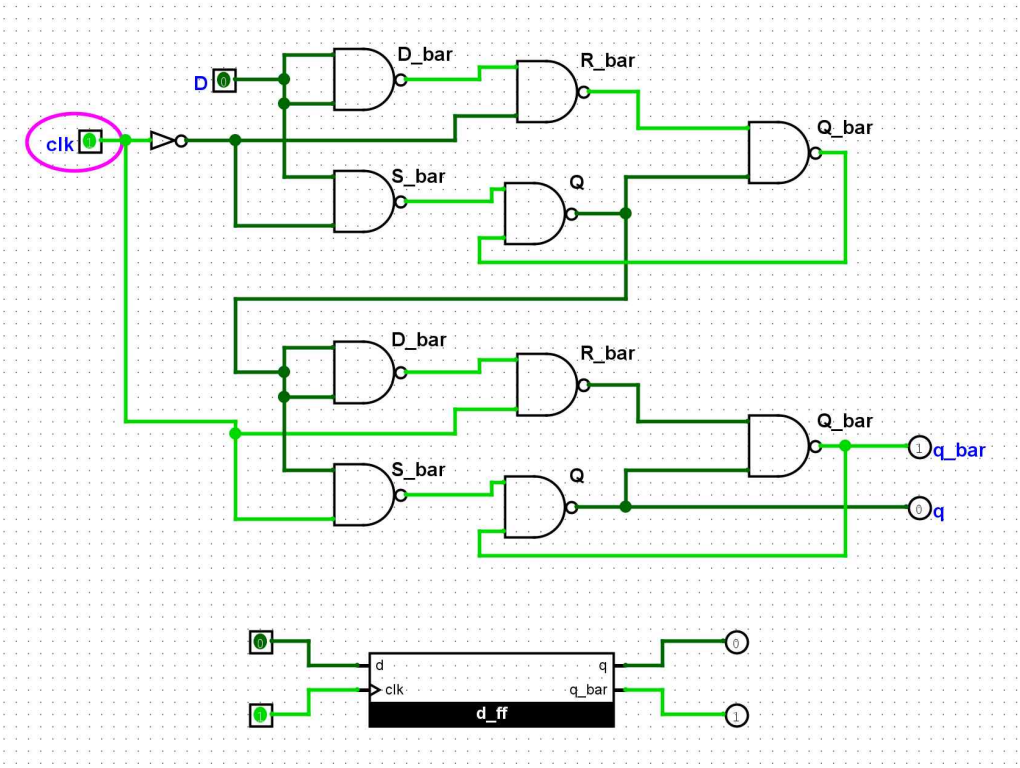


Figure 3-1. clk 0→1 with D=0 → q=0. D=0 is stored on the rising edge

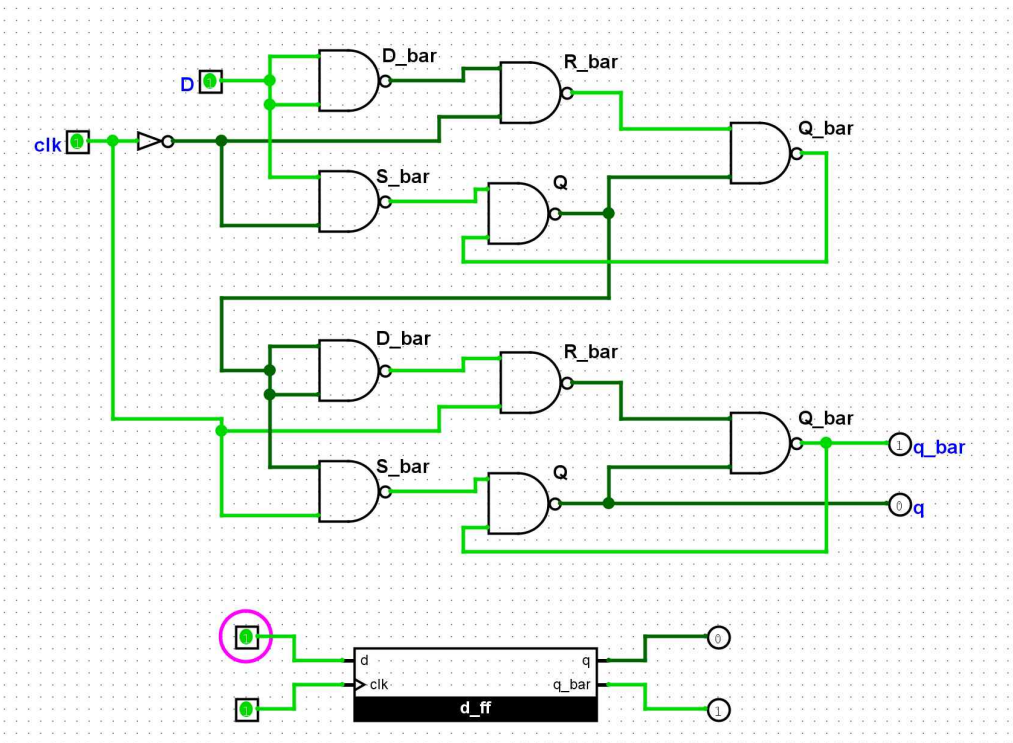


Figure 3-2. clk held at 1, D changed to 1 → q=0 held. No edge, so the output does not change

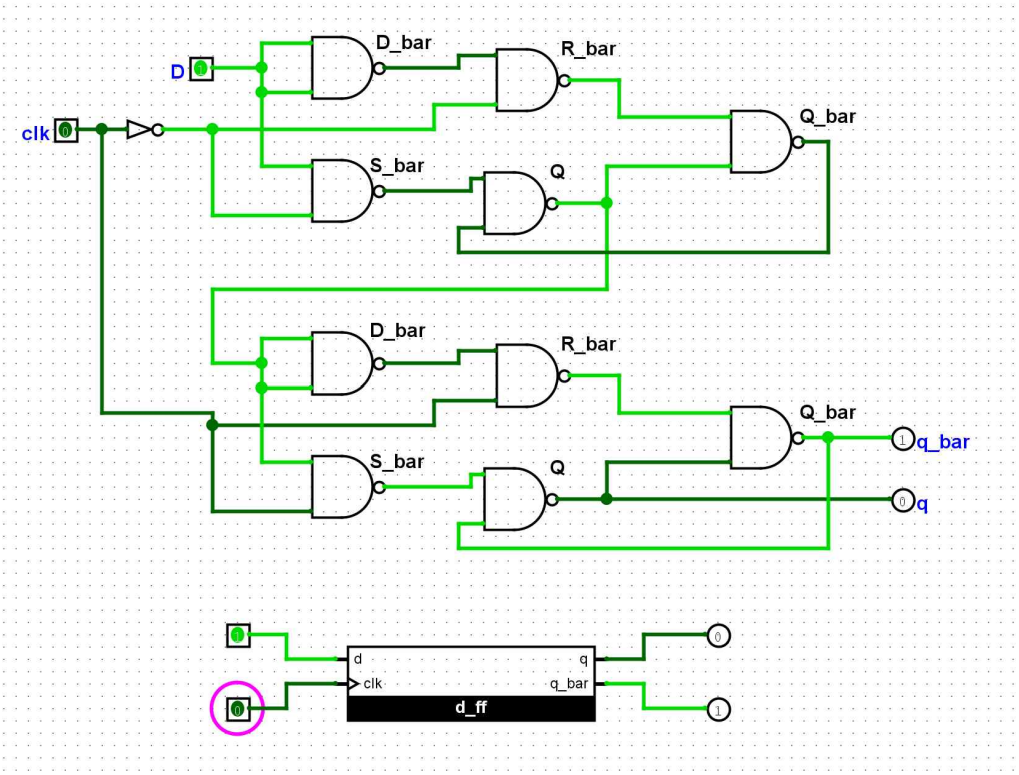


Figure 3-3. clk 1→0 → q=0 held. Only the master accepts D=1; the output stays the same

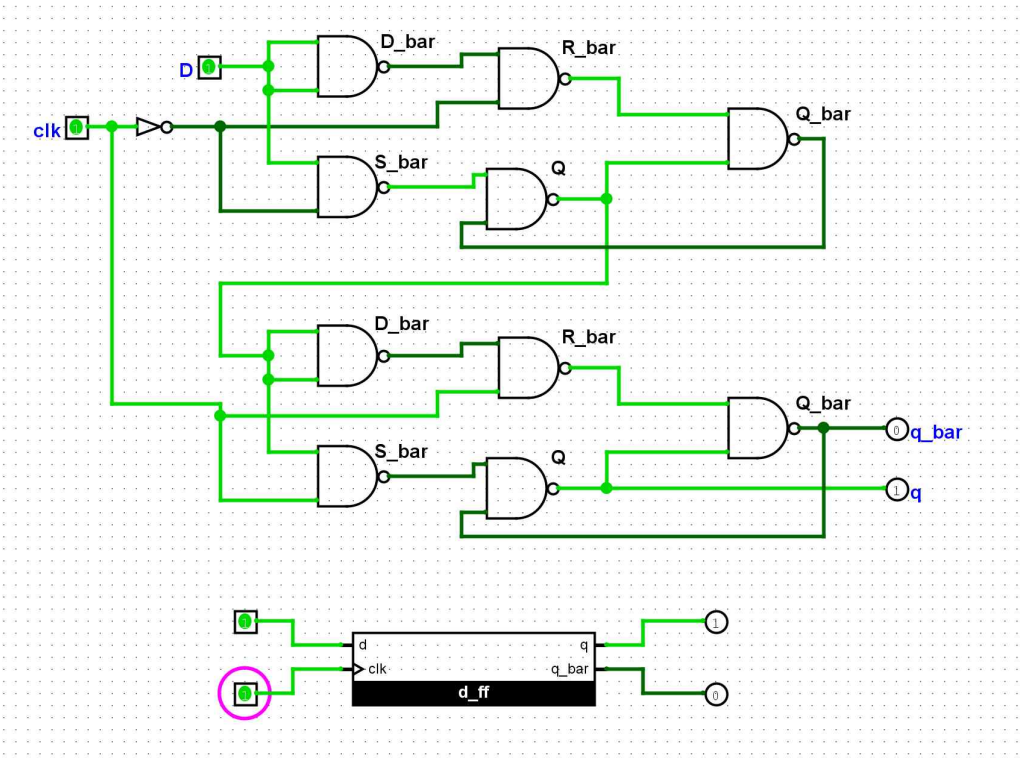


Figure 3-4. clk 0→1 with D=1 → q=1. D=1 is transferred to the output on the rising edge

Unlike the D latch, changing D while clk=1 did not change the output; only the D present at the instant clk changed 0→1 was transferred to the output. The outputs of the two implementations agreed in all four steps.

■ Questa Compilation and Simulation

The same d_ff.vhd was compiled with vcom in Questa Altera Starter FPGA Edition 2025.2 and simulated in vsim, applying the same input sequence as the Logisim experiment with force commands. The script entered in the Transcript window is shown below.

[d_ff_sim.do]

```
add wave sim:/d_ff/d sim:/d_ff/clk sim:/d_ff/q sim:/d_ff/q_bar
force clk 0
force d 0
run 10 ns
force clk 1
run 10 ns
force d 1
run 10 ns
force clk 0
run 10 ns
force clk 1
run 10 ns
wave zoom full
```

```
VSIM 81> vcom {C:\VHDL_Assignment\test2\d_ff.vhd}
# Questa Altera Starter FPGA Edition-64 vcom 2025.2 Compiler 2025.05 May 31 2025
# Start time: 09:46:18 on Nov 11,2026
# vcom -reportprogress 300 C:\VHDL_Assignment\test2\d_ff.vhd
# ** Note: (vcom-220) 'C:/Users/gahyu/AppData/Local/quartus/Ql_rs_latch.mpf' is used as the ini file.
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Compiling entity d_ff
# -- Compiling architecture rtl of d_ff
# End time: 09:46:18 on Nov 11,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
```

Figure 3-5a. Compilation result of d_ff.vhd (Errors: 0, Warnings: 0)

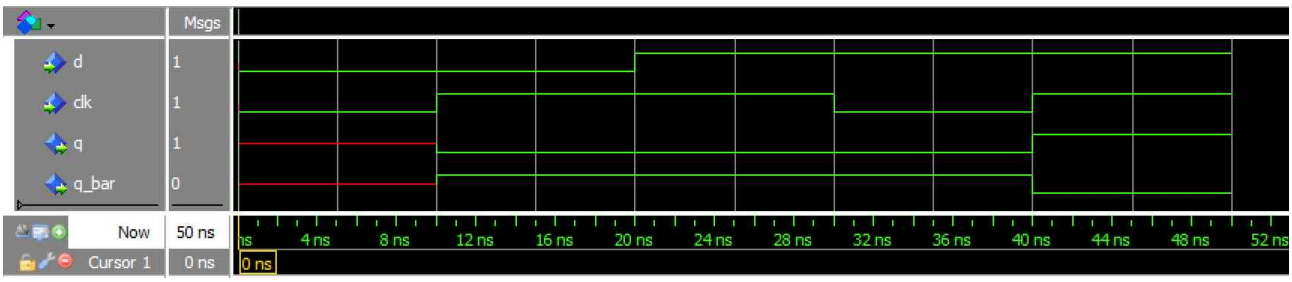


Figure 3-5b. Questa waveform, top to bottom: d, clk, q, q_bar. Each interval is 10 ns

Interval	Inputs	q	Observed behavior
0~10 ns	clk=0, d=0	U	No edge yet; initial value (undefined, red line) held
10 ns	clk 0→1, d=0	0	d=0 stored on the rising edge (Figure 3-1)
20~30 ns	clk=1, d=1	0	No edge, so changing d has no effect (Figure 3-2)
30~40 ns	clk 1→0	0	No change on the falling edge (Figure 3-3)
40 ns	clk 0→1, d=1	1	d=1 stored on the rising edge (Figure 3-4)

q changed only at 10 ns and 40 ns, when clk rose 0→1, and the d changes in between were ignored, matching Figures 3-1 to 3-4. The red line in 0~10 ns is the initial state (U) before the first store; Logisim shows 0 instead.

5. JK Flip-Flop (jk_ff)

NAND gates implementing d_next = J·Q' + K'·Q are placed in front of a D flip-flop and the outputs are fed back to the inputs, giving a flip-flop that holds, Resets, Sets, or toggles on each rising edge.

■ VHDL Code

[jk_ff.vhd]

```
library ieee;
use ieee.std_logic_1164.all;

entity jk_ff is
    port (
        j, k, clk : in std_logic;
        q, q_bar : out std_logic
    );
end jk_ff;

architecture rtl of jk_ff is
    signal qi : std_logic;
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if j = '0' and k = '0' then
                null;
            elsif j = '0' and k = '1' then
                qi <= '0';
            elsif j = '1' and k = '0' then
                qi <= '1';
            elsif j = '1' and k = '1' then
                qi <= not qi;
            end if;
        end if;
    end process;

    q <= qi;
    q_bar <= not qi;
end rtl;
```

■ Code Explanation

- Inside rising_edge(clk), the four J, K combinations are described with an if-elsif chain.
- 00 is null and holds the previous value, 01 Resets, 10 Sets, and 11 toggles with qi <= not

qi.

- This is the d_next logic expression of the gate circuit rewritten as conditional statements; the behavior is the same.

■ Schematic and Results

The upper part of the schematic consists of the input logic that forms k_bar, A = NAND(J, q_bar), B = NAND(k_bar, q), and d_next = NAND(A, B), the Master-Slave D flip-flop behind it, and the feedback lines that return q and q_bar to the input logic. The lower part is the jk_ff VHDL block. J and K were changed and then clk was raised 0→1 while observing the output.

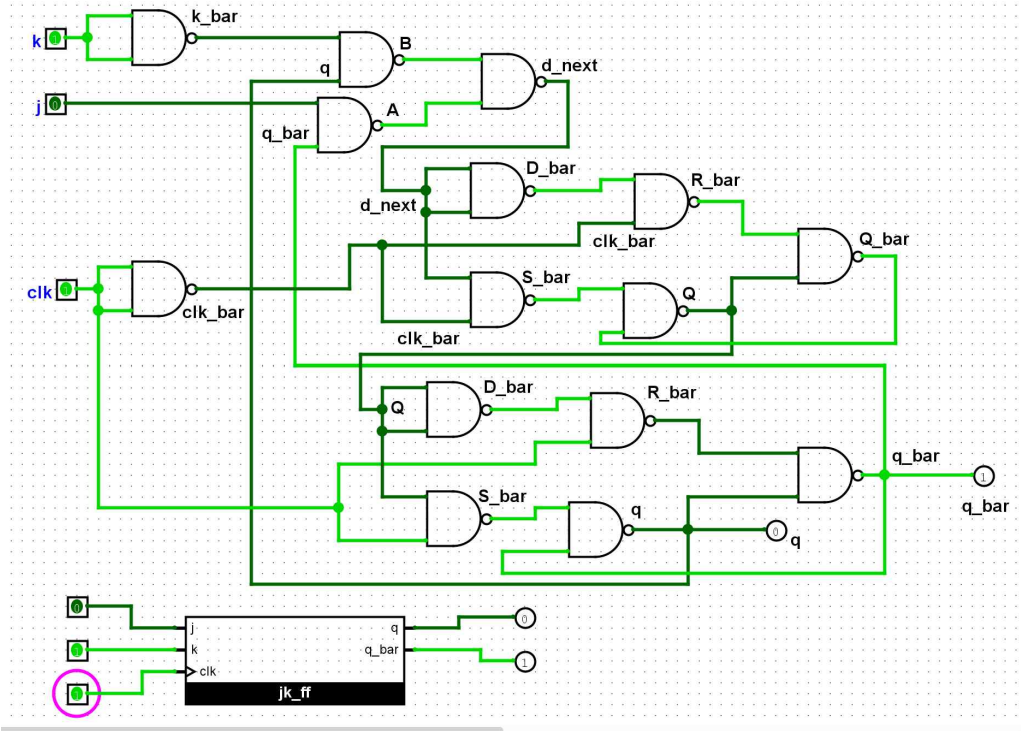


Figure 4-1. J=0, K=1, rising clk edge → q=0 (Reset)

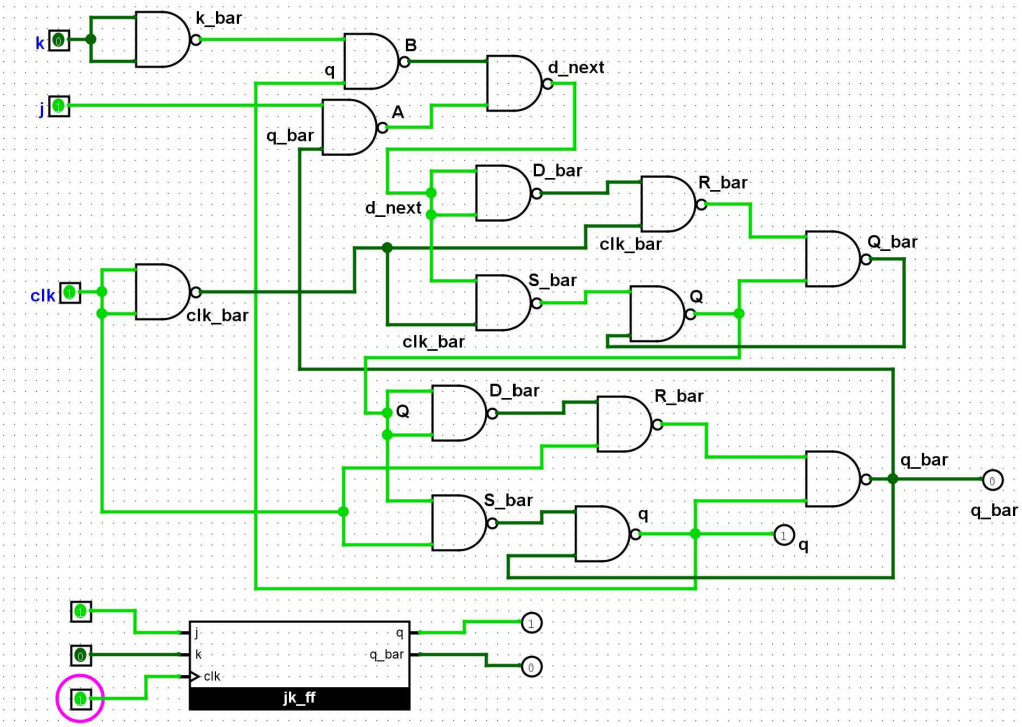


Figure 4-2. J=1, K=0, rising clk edge → q=1 (Set)

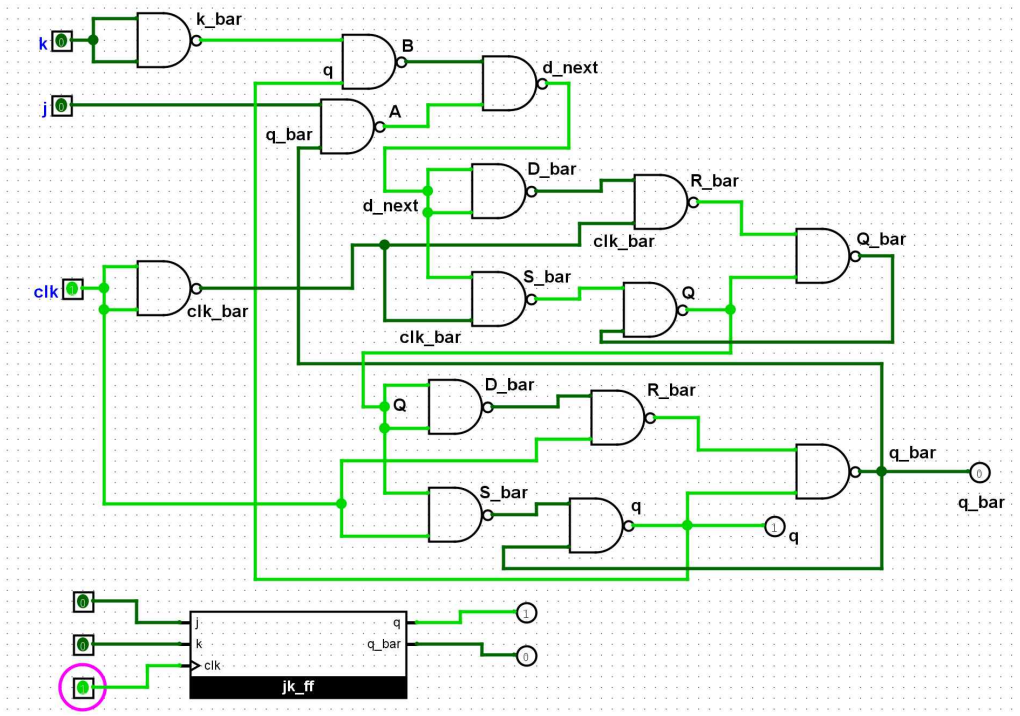


Figure 4-3. J=0, K=0, rising clk edge → q=1 held (Hold)

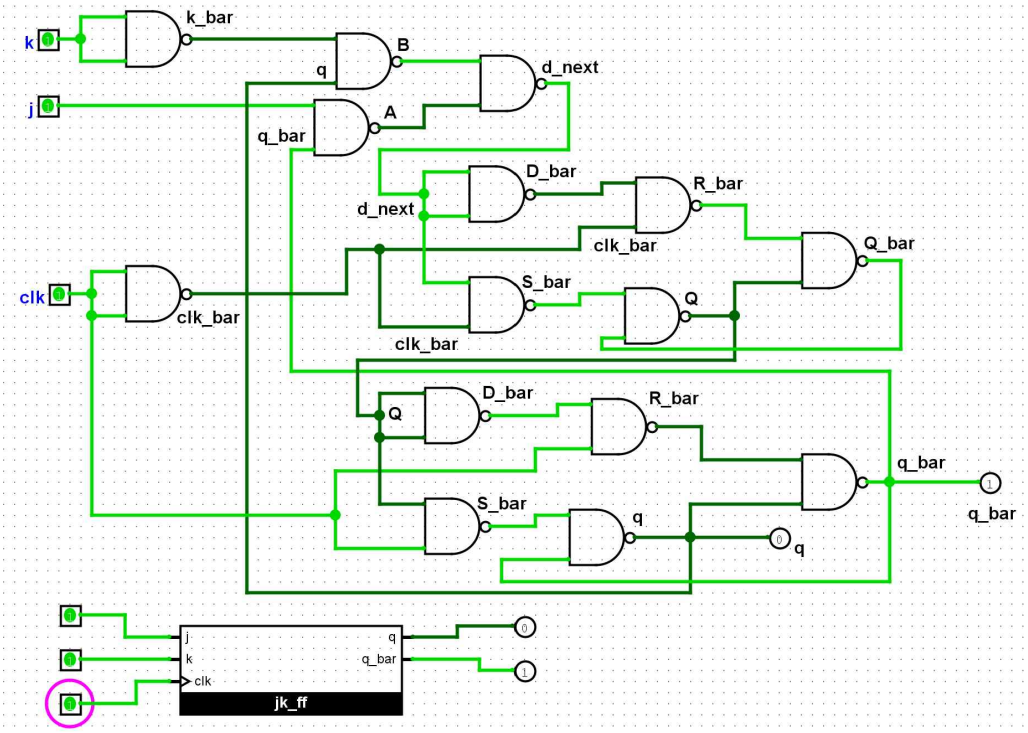


Figure 4-4. J=1, K=1, rising clk edge → q=0 (Toggle, 1→0)

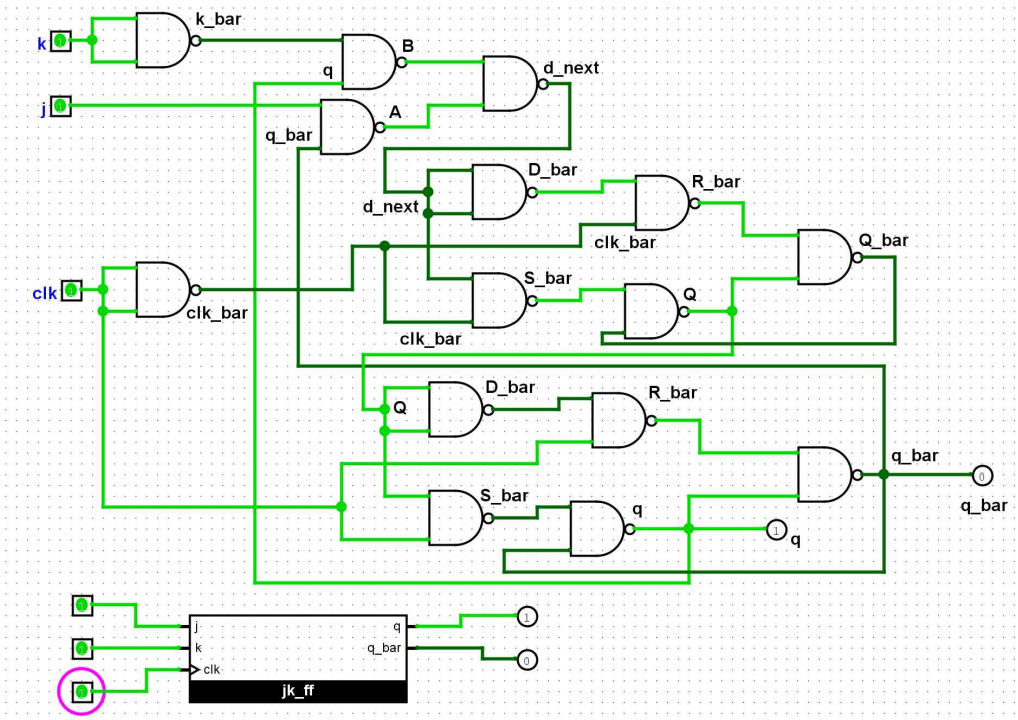


Figure 4-5. J=1, K=1, next rising clk edge → q=1 (Toggle, 0→1)

Reset, Set, Hold, and Toggle all occurred only on the rising clock edge, and with J=K=1 the output inverted on every edge. The outputs of the gate circuit and the VHDL block agreed in all five steps.

■ Questa Compilation and Simulation

The same jk_ff.vhd was compiled with vcom in Questa Altera Starter FPGA Edition 2025.2 and simulated in vsim, applying the same input sequence as the Logisim experiment with force commands. The script entered in the Transcript window is shown below.

[jk_ff_sim.do]

```
add wave sim:/jk_ff/j sim:/jk_ff/k sim:/jk_ff/clk sim:/jk_ff/q sim:/jk_ff/q_bar
force clk 0
force j 0
force k 1
run 10 ns
force clk 1
run 10 ns

force clk 0
force j 1
force k 0
run 10 ns
force clk 1
run 10 ns

force clk 0
force j 0
force k 0
run 10 ns
force clk 1
run 10 ns

force clk 0
force j 1
force k 1
run 10 ns
force clk 1
run 10 ns

force clk 0
run 10 ns
force clk 1
run 10 ns
wave zoom full
```

```
VSIM 114> vcom {C:\VHDL_Assignment\test2\jk_ff.vhd}
# Questa Altera Starter FPGA Edition-64 vcom 2025.2 Compiler 2025.05 May 31 2025
# Start time: 09:49:17 on Nov 11,2026
# vcom -reportprogress 300 C:\VHDL_Assignment\test2\jk_ff.vhd
# ** Note: (vcom-220) 'C:/Users/gahyu/AppData/Local/quartus/Q1_rs_latch.mpf' is used as the ini file.
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Compiling entity jk_ff
# -- Compiling architecture rtl of jk_ff
# End time: 09:49:17 on Nov 11,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
```

Figure 4-6a. Compilation result of jk_ff.vhd (Errors: 0, Warnings: 0)

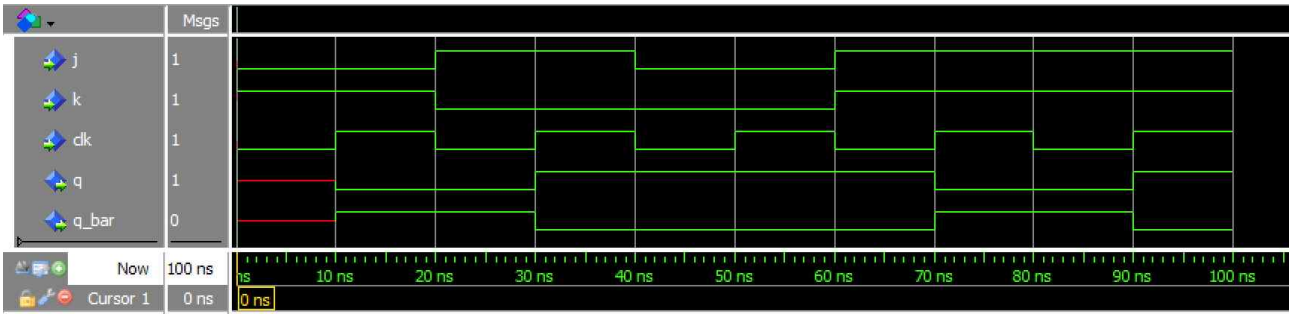


Figure 4-6b. Questa waveform, top to bottom: j, k, clk, q, q_bar. Each interval is 10 ns

Interval	Inputs	q	Observed behavior
10 ns	J=0, K=1, clk 0→1	0	Reset (Figure 4-1)
30 ns	J=1, K=0, clk 0→1	1	Set (Figure 4-2)
50 ns	J=0, K=0, clk 0→1	1	Hold. Previous value kept (Figure 4-3)
70 ns	J=1, K=1, clk 0→1	0	Toggle. Inverted 1→0 (Figure 4-4)
90 ns	J=1, K=1, clk 0→1	1	Toggle. Inverted 0→1 (Figure 4-5)

q changed only at the rising edges (10, 30, 50, 70, 90 ns), operating in the order Reset, Set, Hold, Toggle, Toggle. All of them match the Logisim results in Figures 4-1 to 4-5.

6. D Flip-Flop with Asynchronous Set/Clear (d_ff_async)

The D flip-flop from circuit 3 extended with set and clr inputs that change the output immediately regardless of the clock; when both are 1, Clear has priority.

■ VHDL Code

[d_ff_async.vhd]

```
library ieee;
use ieee.std_logic_1164.all;

entity d_ff_async is
    port (
        d, clk, set, clr : in std_logic;
        q, q_bar : out std_logic
    );
end d_ff_async;

architecture rtl of d_ff_async is
    signal qi : std_logic;
begin
    process(clk, set, clr)
    begin
        if clr = '1' then
            qi <= '0';
        elsif set = '1' then
            qi <= '1';
        elsif rising_edge(clk) then
            qi <= d;
        end if;
    end process;

    q <= qi;
    q_bar <= not qi;
end rtl;
```

■ Code Explanation

- process(clk, set, clr) puts set and clr in the sensitivity list so that the flip-flop responds immediately without a clock.
- if clr = '1' is checked first, so Clear has priority when set and clr are 1 at the same time.
- Only when both are 0 does control fall through to elsif rising_edge(clk), where it behaves

like an ordinary D flip-flop.

■ Schematic and Results

The upper part of the schematic is the two-stage master/slave D flip-flop with the added paths ASYNC = set OR clr and SET_eff = set AND clr_bar. When ASYNC is 1, the clock-synchronous paths (SD, RD) are blocked and S_bar and R_bar are determined directly by set/clr. The lower part is the d_ff_async VHDL block. First, with the clock held at 0, only clr and set were changed to observe the asynchronous behavior; then both were lowered to 0 and the clocked behavior was checked.

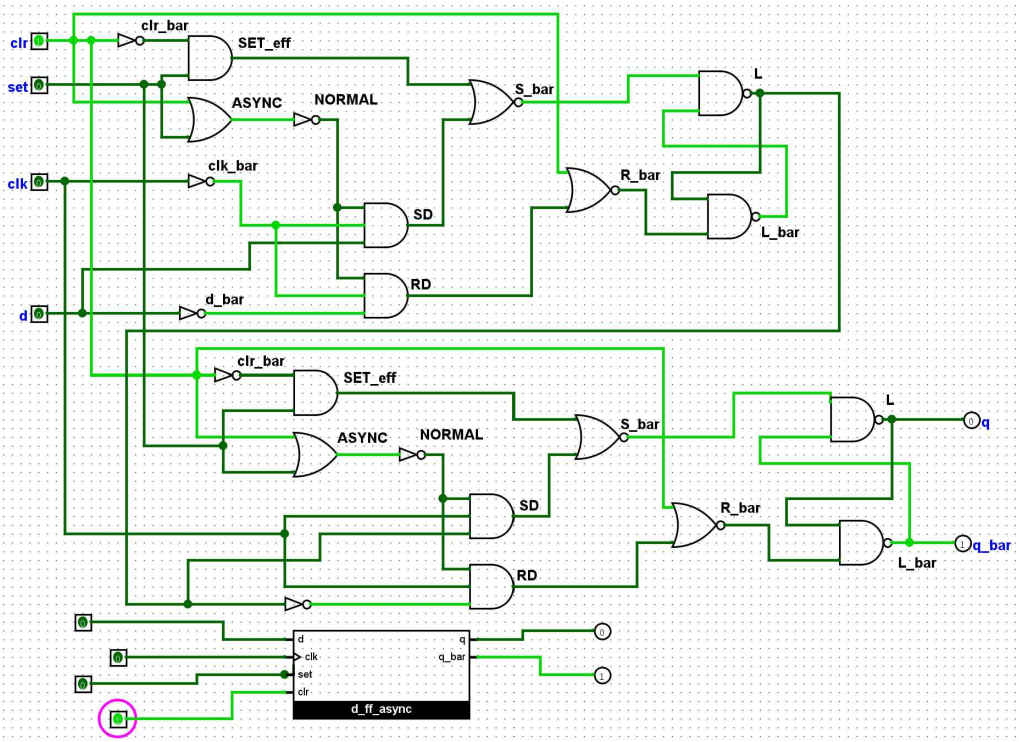


Figure 5-1. clr=1, set=0, clk=0 → q=0 immediately (asynchronous Clear)

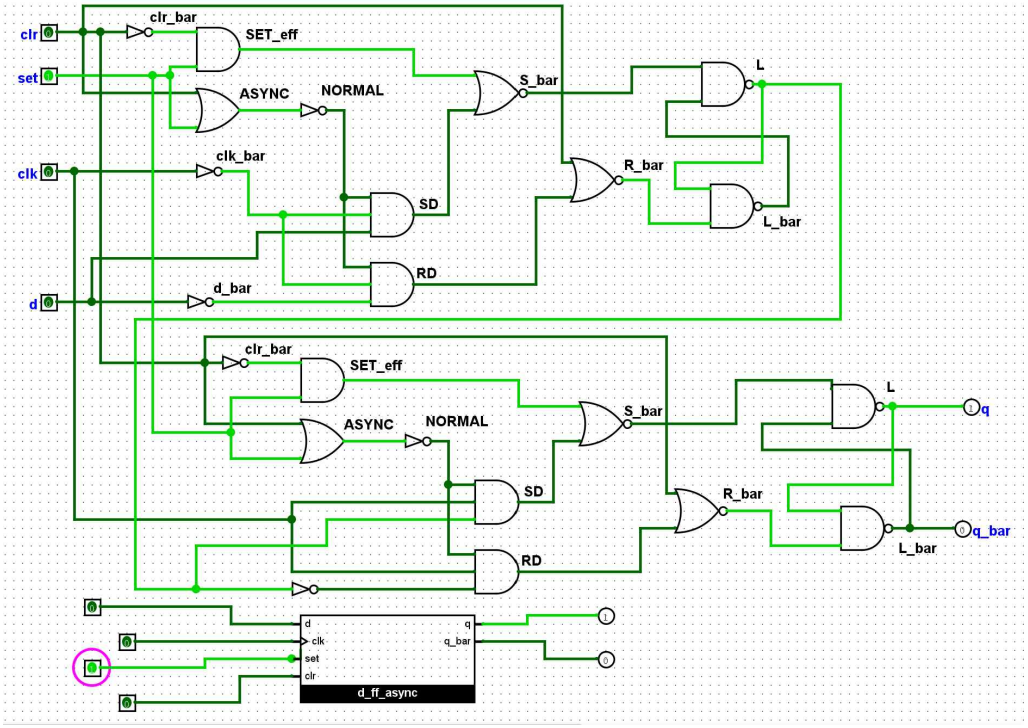


Figure 5-2. clr=0, set=1, clk=0 → q=1 immediately (asynchronous Set)

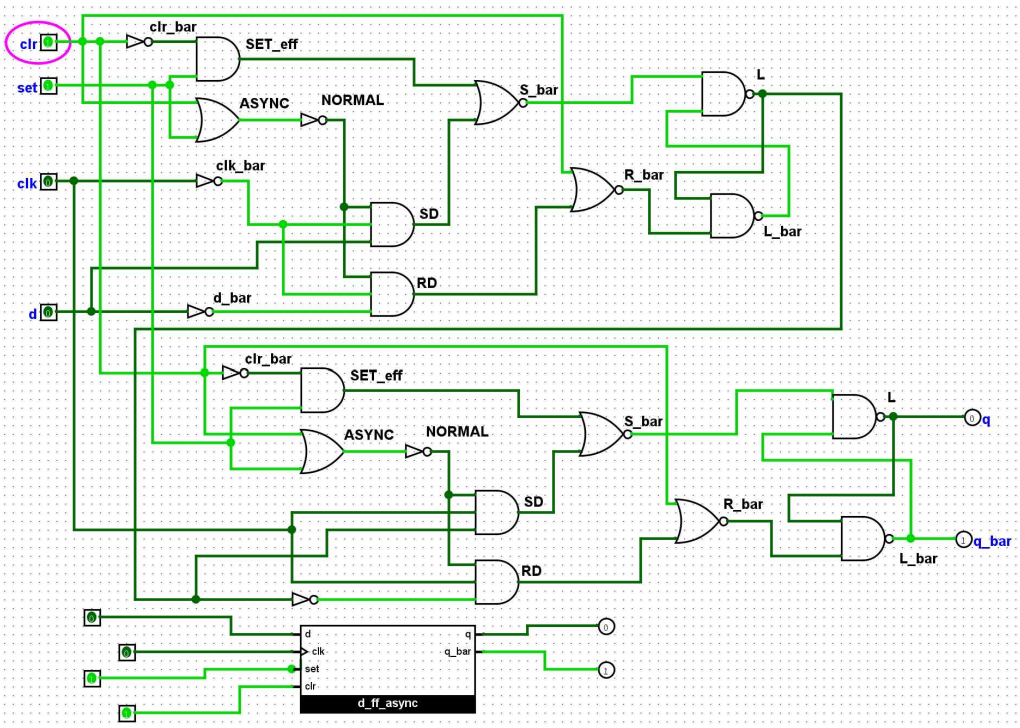


Figure 5-3. clr=1, set=1 → q=0 (Clear has priority)

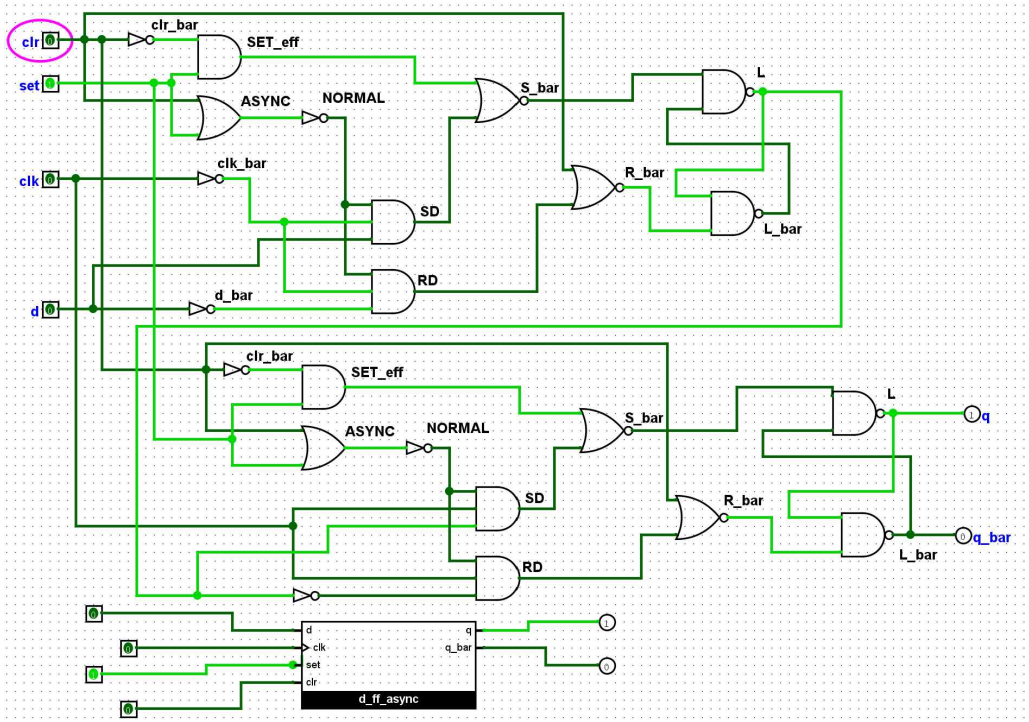


Figure 5-4. clr=0, set=1 → q=1 (Set is applied again once clr is lowered)

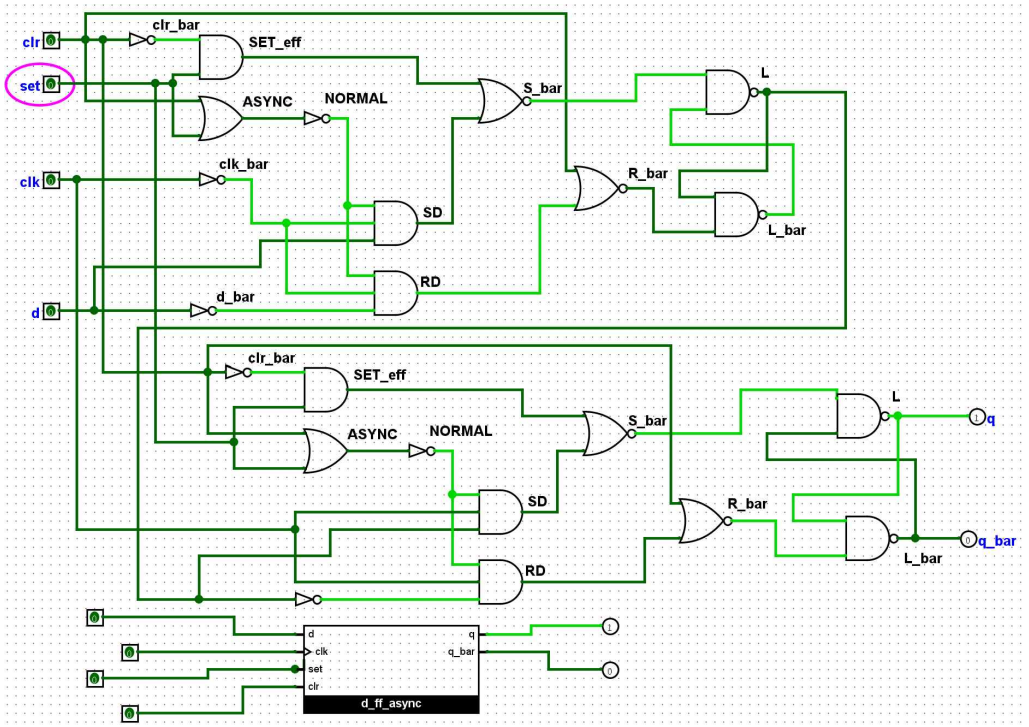


Figure 5-5. set=0, clr=0 → q=1 held (asynchronous inputs released, value preserved)

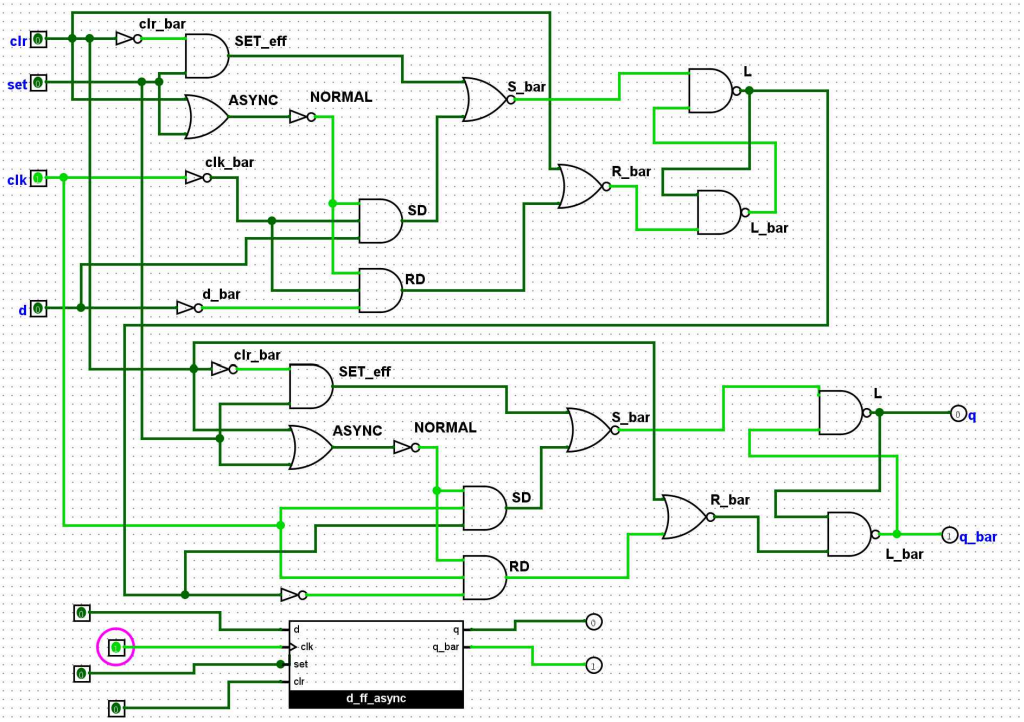


Figure 5-6. d=0, clk 0→1 → q=0 (d stored on the rising edge like an ordinary D flip-flop)

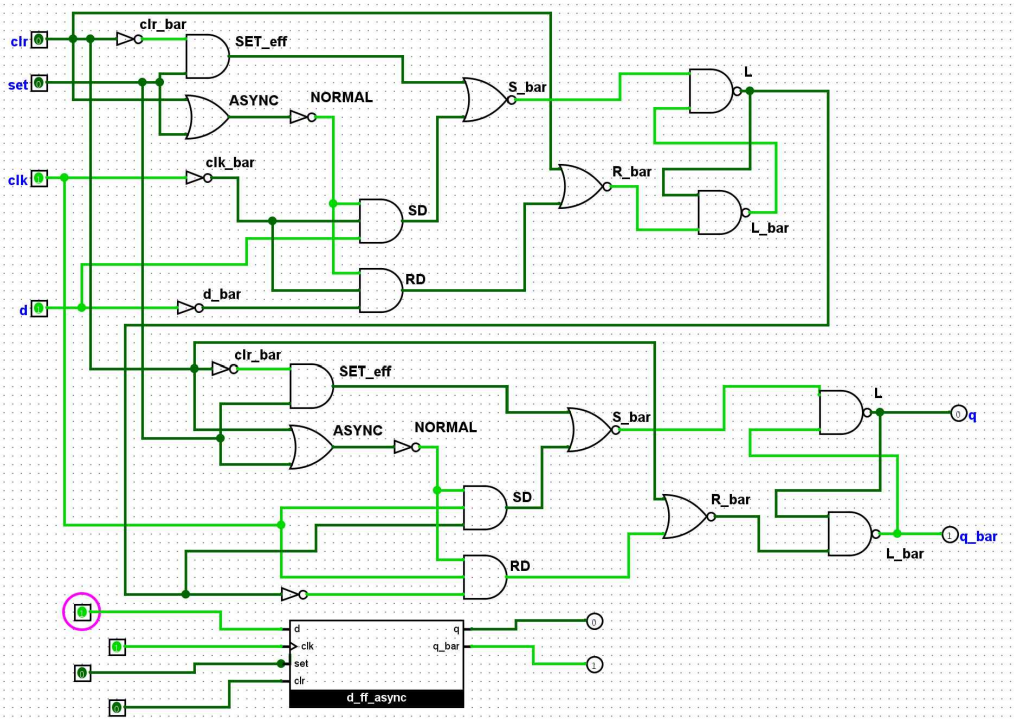


Figure 5-7. clk held at 1, d changed to 1 → q=0 held (no edge, so no change)

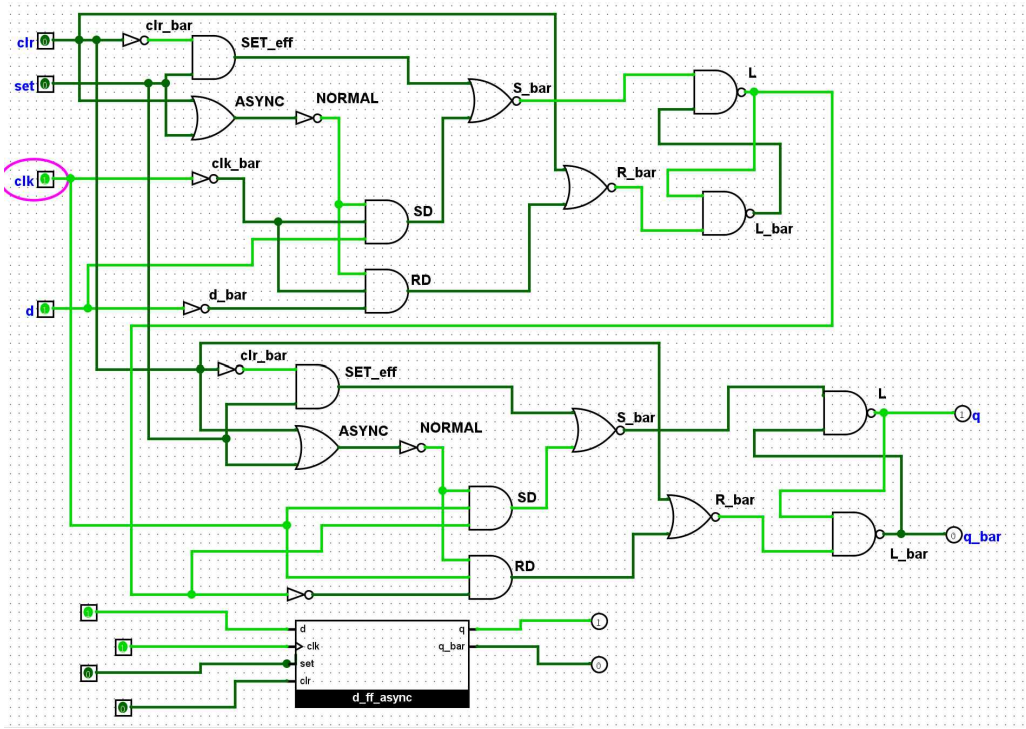


Figure 5-8. d=1, clk 0→1 → q=1 (d=1 stored on the rising edge)

clr and set changed the output immediately even while the clock was 0, and Clear had priority when both were 1. After both asynchronous inputs were lowered, d was stored only on the rising edge as in circuit 3, and the outputs of the two implementations agreed in all eight steps.

■ Questa Compilation and Simulation

The same d_ff_async.vhd was compiled with vcom in Questa Altera Starter FPGA Edition 2025.2 and simulated in vsim, applying the same input sequence as the Logisim experiment with force commands. The script entered in the Transcript window is shown below.

[d_ff_async_sim.do]

```
add wave sim:/d_ff_async/d sim:/d_ff_async/clk sim:/d_ff_async/set sim:/d_ff_async/clr
sim:/d_ff_async/q sim:/d_ff_async/q_bar
force clk 0
force d 0
force set 0
force clr 1
run 10 ns
force clr 0
force set 1
run 10 ns
```

```
force clr 1
run 10 ns
force clr 0
run 10 ns
force set 0
run 10 ns
force clk 1
run 10 ns
force d 1
run 10 ns
force clk 0
run 10 ns
force clk 1
run 10 ns
wave zoom full
```

```
VSIM 141> vcom {C:\VHDL_Assignment\test2\d_ff_async.vhd}
# Questa Altera Starter FPGA Edition-64 vcom 2025.2 Compiler 2025.05 May 31 2025
# Start time: 09:51:20 on Nov 11,2026
# vcom -reportprogress 300 C:\VHDL_Assignment\test2\d_ff_async.vhd
# ** Note: (vcom-220) 'C:/Users/gahyu/AppData/Local/quartus/Ql_rs_latch.mpf' is used as the ini file.
# -- Loading package STANDARD
# -- Loading package TEXTIO
# -- Loading package std_logic_1164
# -- Compiling entity d_ff_async
# -- Compiling architecture rtl of d_ff_async
# End time: 09:51:20 on Nov 11,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
```

Figure 5-9a. Compilation result of d_ff_async.vhd (Errors: 0, Warnings: 0)

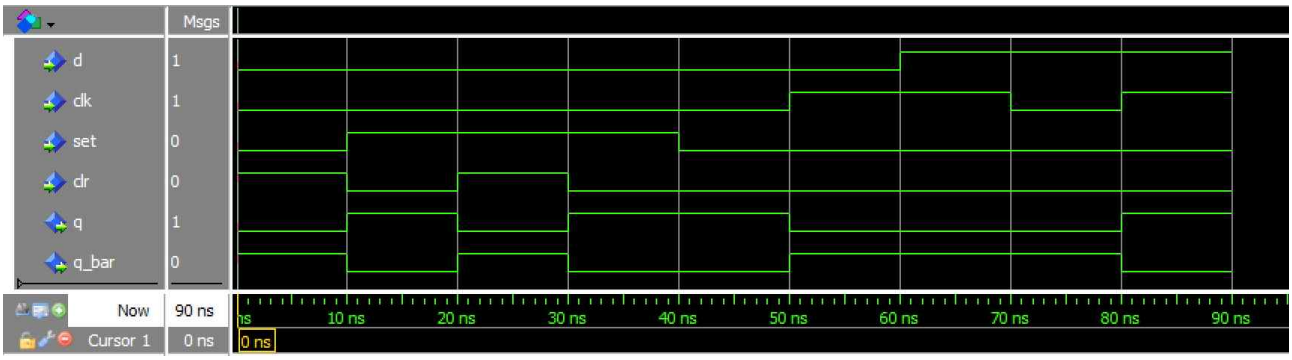


Figure 5-9b. Questa waveform, top to bottom: d, clk, set, clr, q, q_bar. Each interval is 10 ns

Interval	Inputs	q	Observed behavior
0~10 ns	clr=1, set=0, clk=0	0	Asynchronous Clear. 0 immediately, no clock (Figure 5-1)
10~20 ns	clr=0, set=1	1	Asynchronous Set. 1 immediately (Figure 5-2)
20~30 ns	clr=1, set=1	0	Clear has priority when both are 1 (Figure 5-3)
30~40 ns	clr=0, set=1	1	Set applied again once clr is lowered (Figure 5-4)
40~50 ns	set=0, clr=0	1	Asynchronous inputs released, value kept (Figure 5-5)
50 ns	clk 0→1, d=0	0	d=0 stored on the edge, as a normal D FF (Figure 5-6)
60~70 ns	clk=1, d=1	0	No edge, so no change (Figure 5-7)
80 ns	clk 0→1, d=1	1	d=1 stored on the edge (Figure 5-8)

clr and set changed q immediately even with clk=0, Clear won when both were 1, and after both were lowered d was stored only on the rising edge. All eight intervals match Figures 5-1 to 5-8.

7. Overall Comparison

The results of implementing and testing each circuit in three ways are summarized below. For every circuit, the Logisim gate circuit, the Logisim VHDL block, and the Questa simulation produced identical outputs for the same input sequence.

Circuit	Logisim Gate Circuit	Logisim VHDL Block	Questa (vcom/vsim)	Result
RS latch	Two cross-coupled NOR gates	rs_latch.vhd	Errors 0, 5-interval waveform	Match
D latch	Four NAND gates	d_latch.vhd	Errors 0, 5-interval waveform	Match
D flip-flop	Two D latches + NOT (Master-Slave)	d_ff.vhd	Errors 0, 2 rising edges	Match
JK flip-flop	NAND input logic + Master-Slave + feedback	jk_ff.vhd	Errors 0, 5 rising edges	Match
Async Set/Clear D FF	D FF + asynchronous set/clr paths	d_ff_async.vhd	Errors 0, 8-interval waveform	Match

- The gate circuits create the memory function with feedback wires, while VHDL describes the same function with concurrent assignments (RS latch) or if conditions inside a process (the others).
- In the Questa waveforms the initial value of a flip-flop is shown as U (red line) because nothing has been stored before the first rising edge; the behavior after the first edge is the same as in Logisim.
- The submission repository contains the five .vhd files, five Logisim .circ files, five Questa simulation scripts (.do), and the PDF of this report.